

Problem Statement:-

Build a music playlist manager using linked lists to manage song order, insertions, deletions, and shuffle functionalities.

Solution:-

//Build a music playlist manager using linked lists to manage song order, insertions, deletions, and shuffle functionalities.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
struct Song {
    char title[50];
    char artist[50];
    struct Song* next;
};
struct Song* createSong(char* title, char* artist) {
    struct Song* newSong = (struct Song*)malloc(sizeof(struct Song));
    strcpy(newSong->title, title);
    strcpy(newSong->artist, artist);
    newSong->next = NULL;
    return newSong;
}
void addSong(struct Song** head, char* title, char* artist) {
    struct Song* newSong = createSong(title, artist);
    if (*head == NULL) {
        *head = newSong;
    } else {
        struct Song* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newSong;
    }
    printf("Song \"%s\" by %s added to the playlist.\n", title, artist);
}
void deleteSong(struct Song** head, char* title) {
    struct Song* temp = *head;
    struct Song* prev = NULL;
    if (temp != NULL && strcmp(temp->title, title) == 0) {
        *head = temp->next;
```

```

        free(temp);
        printf("Song \"%s\" deleted from the playlist.\n", title);
        return;
    }
    while (temp != NULL && strcmp(temp->title, title) != 0) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Song \"%s\" not found in the playlist.\n", title);
        return;
    }
    prev->next = temp->next;
    free(temp);
    printf("Song \"%s\" deleted from the playlist.\n", title);
}

void displayPlaylist(struct Song* head) {
    if (head == NULL) {
        printf("The playlist is empty.\n");
        return;
    }
    struct Song* temp = head;
    printf("\nMusic Playlist:\n");
    while (temp != NULL) {
        printf("Title: %s, Artist: %s\n", temp->title, temp->artist);
        temp = temp->next;
    }
}

int countSongs(struct Song* head) {
    int count = 0;
    struct Song* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

void shufflePlaylist(struct Song** head) {
    int count = countSongs(*head);
    if (count < 2) {
        printf("Not enough songs to shuffle.\n");
        return;
    }

    srand(time(NULL));

```

```

    struct Song* temp = *head;
    struct Song* prev = NULL;
    for (int i = 0; i < count * 3; i++) {
        int r = rand() % count;
        temp = *head;
        prev = NULL;
        for (int j = 0; j < r; j++) {
            prev = temp;
            temp = temp->next;
        }
        if (prev != NULL) {
            prev->next = temp->next;
            temp->next = *head;
            *head = temp;
        }
    }
    printf("Playlist shuffled.\n");
}

void freePlaylist(struct Song* head) {
    struct Song* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    struct Song* playlist = NULL;
    int choice;
    char title[50];
    char artist[50];

    while (1) {
        printf("\n--- Music Playlist Manager ---\n");
        printf("1. Add Song\n");
        printf("2. Delete Song\n");
        printf("3. Display Playlist\n");
        printf("4. Shuffle Playlist\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter song title: ");

```

```

        scanf(" %[^\\n]s", title);
        printf("Enter artist name: ");
        scanf(" %[^\\n]s", artist);
        addSong(&playlist, title, artist);
        break;
    case 2:
        printf("Enter song title to delete: ");
        scanf(" %[^\\n]s", title);
        deleteSong(&playlist, title);
        break;
    case 3:
        displayPlaylist(playlist);
        break;
    case 4:
        shufflePlaylist(&playlist);
        break;
    case 5:
        freePlaylist(playlist);
        printf("Exiting Playlist Manager.\\n");
        return 0;
    default:
        printf("Invalid choice. Please try again.\\n");
    }
}
}

```

Output

TIMELINE OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

Code + - [] ... X

```
PS C:\Users\harsh\OneDrive\Desktop\C++\Ternary operator> cd "c:\Users\harsh\OneDrive\Desktop\C++\Ternary operator\" ; if ($?) { gcc harsh.c -o harsh } ; if ($?) { .\harsh }
```

--- Music Playlist Manager ---

1. Add Song
2. Delete Song
3. Display Playlist
4. Shuffle Playlist
5. Exit

Enter your choice:

TIMELINE OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

Code + - [] ... X

```
PS C:\Users\harsh\OneDrive\Desktop\C++\Ternary operator> cd "c:\Users\harsh\OneDrive\Desktop\C++\Ternary operator\" ; if ($?) { gcc harsh.c -o harsh } ; if ($?) { .\harsh }
```

--- Music Playlist Manager ---

1. Add Song
2. Delete Song
3. Display Playlist
4. Shuffle Playlist
5. Exit

Enter your choice: 1

Enter song title: Ehssan Tera Hoga Mujh par

Enter artist name: Mohammed Rafi

Song "Ehssan Tera Hoga Mujh par" by Mohammed Rafi added to the playlist.

TIMELINE OUTPUT DEBUG CONSOLE TERMINAL PORTS SEARCH ERROR

Code + - [] ... X

1. Add Song
2. Delete Song
3. Display Playlist
4. Shuffle Playlist
5. Exit

Enter your choice: Likhe Jo Khat Tujhe

Enter song title: Enter artist name: Mohammed Rafi

Song "Likhe Jo Khat Tujhe" by Mohammed Rafi added to the playlist.

3. Display Playlist

4. Shuffle Playlist

5. Exit

Enter your choice: 3

Music Playlist:

Title: Ehsan Tera Hoga Mujh par, Artist: Mohammed Rafi

Title: Likhe Jo Khat Tujhe, Artist: Mohammed Rafi

Title: Chura Liya Hai Tumne Jo Dil Ko, Artist: Mohammed Rafi

Result

When you run the playlist manager program, the following actions can be performed:

1. **Add Song:**
 - You can input a song title and artist, and the song will be added to the playlist.
 - The program confirms the addition with a message.
2. **Delete Song:**
 - You can delete a song by its title.
 - If the song is found, it will be removed, and you'll get a confirmation message.
 - If the song is not found, an error message will be shown.
3. **Display Playlist:**
 - Displays all songs currently in the playlist with their respective title and artist.
 - If the playlist is empty, a message indicating so will be printed.
4. **Shuffle Playlist:**
 - Randomly reorders the songs in the playlist using a basic shuffling algorithm.
 - If there are fewer than two songs, the program will notify you that shuffling is not possible.
5. **Exit:**
 - Exits the program and frees all dynamically allocated memory.